Сессионное задание

Вопрос 1. Опишите трехсторонний протокол распределения ключей между парами участников с использованием третьей стороны

Опишем классический протокол с доверенным центром распределения ключей (KDC) на примере базовой схемы Kerberos. Он иллюстрирует, как третья сторона помогает двум участникам A и B установить общий сеансовый ключ.

Предпосылки и нотация

- A - инициатор (клиент), B - респондент (сервер/служба), T - доверенная третья сторона (KDC).
- K_AT и K_BT - долговременные симметричные ключи A↔T и B↔T соответственно.
- k - свежий сеансовый ключ A↔B; L - срок его действия.
- N_A - одноразовое число (nonce); T_A - метка времени A.
- E_K(X) - шифрование сообщения X на ключе K.

Роли сторон

- A: инициатор, запрашивает у центра распределения ключей сеансовый ключ для связи с B.
- B: второй участник, получает билет и извлекает из него сеансовый ключ.
- T (KDC): доверенный центр, генерирует сеансовый ключ и безопасно передаёт его участникам.

Этапы протокола (кто, кому и что отправляет)

1. Запрос A к T

A → T: A, B, N_A

Назначение: A сообщает требуемого корреспондента B и добавляет одноразовый N_A для защиты от подмен/повторов ответа.

2. Ответ T → A

T → A: ticket_B, E_{K_AT}(k, N_A, L, B)

Действия T: сгенерировать свежий k и срок L; подготовить «билет» для B:

- ticket_B = E_{K_BT}(k, A, L)
- для A зашифровать E_{K_AT}(k, N_A, L, B)

Действия A: расшифровать свою часть на K_AT, проверить N_A (свежесть и адресность ответа), запомнить k и L.

3. Передача билета B и аутентификатора

A → B: ticket_B, authenticator

где authenticator = E_k(A, T_A)

Действия B: расшифровать ticket_B ключом K_BT, получить k и L; затем проверить authenticator на k: совпадение имени A и допустимость метки T_A (нет повтора/просрочки).

4. Взаимная аутентификация (опционально)

B → A: E_k(T_A)

Назначение: доказать A владение тем же k и завершить взаимную аутентификацию.

Итог
Участники A и B устанавливают общий сеансовый ключ k, сгенерированный доверенным центром T и переданный им в зашифрованном виде.

Свойства и замечания

- сеансовый ключ не передаётся в открытом виде;
- одноразовое число и метка времени позволяют проверять свежесть сообщений и защищаться от повторных атак.

Источник: Menezes, A., Oorshot, P., Vanstone, S. Handbook of Applied Cryptography [Text] / A.J. Menezes, P.C. Oorshot. 5th edition. - USA.: CRC Press, 2001, §12.3.2, Protocol 12.24 (simplified). Для уточнения отдельных моментов также сверялась с книгой Шнайера «Прикладная криптография». Для ответа использовались учебники из списков литературы, которые представлены в учебных модулях Я.Практикума

Вопрос 2. Приведите формальные модели шифров

Нотация и обозначения (едины для всего раздела)
- M - множество открытых текстов; C - множество шифртекстов; K - множество ключей.
- Gen - генерация ключей; Enc/Dec - шифрование/расшифрование; E_k, D_k - алгоритмы при фиксированном ключе k.
- pk/sk - открытый/закрытый ключ; IV - инициализирующий вектор; r - случайность; $\oplus$ - побитовое XOR.
- Корректность: D_k(Enc_k(m)) = m для всех m $\in$ M и допустимых k $\in$ K.

1. Общая формальная модель криптосистемы
- Формула: S = (M, C, K, E, D); для каждого k $\in$ K заданы E_k: M→C и D_k: C→M, причём D_k(E_k(m)) = m.
- Компоненты - назначение:
- M, C - пространства сообщений и шифртекстов.
- K - пространство ключей (выбирает конкретное преобразование).
- E, D - семейства алгоритмов шифрования/расшифрования по ключу.
- Назначение: базовая абстракция для описания любых шифров и их корректности.

2. Симметричная схема шифрования (SKE)

- Формула: Π = (Gen, Enc, Dec); k ← Gen(), c = Enc_k(m), m = Dec_k(c); корректность как выше.
- Компоненты - назначение:
- k - общий секрет отправителя и получателя.
- Enc_k / Dec_k - преобразуют сообщение в шифртекст и обратно.
- m, c - открытый текст и шифртекст.

- Назначение: защита данных при наличии заранее общего секрета; база для блочных и поточных шифров.

3. Асимметричная схема шифрования (PKE)

- Формула: Π = (Gen, Enc, Dec); (pk, sk) ← Gen(), c = Enc_pk(m), m = Dec_sk(c); Dec_sk(Enc_pk(m)) = m.
- Компоненты - назначение:
- pk - открытый ключ для шифрования; sk - секретный ключ для расшифрования.
- Gen - создаёт согласованную пару ключей.
- Enc_pk / Dec_sk - алгоритмы по соответствующим ключам.
- Назначение: защищённая передача без предварительного общего секрета; основа обмена ключами и протоколов.

4. Блочный шифр как семейство перестановок (PRP-модель)

- Формула: для фиксированного размера блока n, {E_k}_{k∈K} - семейство перестановок на {0,1}^n, D_k = E_k^{-1}.
- Компоненты - назначение:
- n, {0,1}^n - размер и пространство блоков.
- k ∈ K - выбирает конкретную перестановку E_k.
- D_k - обратное преобразование для расшифрования.
- Назначение: строгая модель блочного шифра; для сообщений произвольной длины используются режимы (CBC, CTR, GCM и др.).

5. Поточный шифр

- Формула: z = G(k, IV), c = m ⊕ z, m = c ⊕ z.
- Компоненты - назначение:
- k - секретный ключ; IV/nonce - обеспечивает различие потоков.
- G - генератор ключевого потока (псевдослучайная гамма).
- z - ключевой поток; ⊕ - операция шифрования/дешифрования.
- Назначение: шифрование потоков/данных произвольной длины с требованиями к уникальности IV.

6. Одноразовый блокнот (OTP)

- Формула: M = C = K = {0,1}^n, c = m ⊕ k, m = c ⊕ k.
- Компоненты - назначение:
- k - истинно случайный, длиной с сообщение, используется один раз и хранится в секрете.
- m, c - открытый текст и шифртекст; ⊕ - операция сложения по модулю 2.
- Назначение: теоретическая модель абсолютно стойкого шифра при строгом соблюдении условий.

7. Детерминированное и вероятностное шифрование

- Детерминированное: c = Enc_k(m) - одинаковые m при одном k дают одинаковые c.
- Вероятностное: c ← Enc_k(m; r) - вводится случайность r/IV, что скрывает повторы сообщений.

- Компоненты - назначение:
- r/IV - свежесть/рандомизация; Enc использует её при шифровании.
- Назначение: вероятностные схемы обеспечивают стойкость уровня IND-CPA и выше; детерминированные применяют ограниченно.

8. Модель шифра с точки зрения стойкости (эксперименты)

- IND-CPA (выбор сообщения): противник выбирает m0, m1, получает шифрование одного из них и пытается угадать бит выбора; преимущество должно быть пренебрежимо мало.
- IND-CCA (адаптивный доступ к оракулу расшифрования): противник имеет доступ к Dec(·) с исключением целевого шифртекста (CCA2 - до и после вызова вызова шифрования-вызова).
- PRG/PRF/PRP: определения стохастических примитивов; потоковые шифры строятся на PRG, блочные - на PRP.
- Назначение: формальные критерии безопасности и сопоставление возможностей нарушителя.

Список литературы

- Menezes, A., Oorshot, P., Vanstone, S. Handbook of Applied Cryptography [Text] / A.J. Menezes, P.C. Oorshot. 5th edition. - USA.: CRC Press, 2001.
- Mao, W. Modern Cryptography: Theory and Practice. Prentice Hall, 2004 (рус. пер. «Современная криптография», 2005).
- Smart, N. Cryptography: Theory and Practice. Chapman & Hall/CRC, 4th ed., 2016 (рус. пер. «Криптография», 2005).

Вопрос 3. Приведите принципы построения поточных систем шифрования

Общая модель поточного шифра

1. Базовая схема потокового шифрования:

$z = G(k, IV)$

$c = m \oplus z$

$m = c \oplus z$

Обозначения: m - открытый текст; c - шифртекст; k - секретный ключ; IV (или nonce) - инициализирующий вектор/уникальный начальный параметр; G - генератор ключевого потока; z - ключевой поток (гамма); $\oplus$ - побитовое XOR.

Смысл: отправитель и получатель, имея один и тот же k и согласованный IV, порождают одинаковый z и, соответственно, корректно шифруют/расшифруют поток данных.

2. Принципы построения поточных систем шифрования

2.1. Принцип формирования ключевого потока

Генератор ключевого потока вырабатывает последовательность битов/байтов, которая используется для шифрования открытого текста.

Формально: $z_1, z_2, \ldots, z_n = G(k, IV)$; каждый элемент шифруется как $c_i = m_i \oplus z_i$.

Назначение: обеспечивает потоковую обработку данных по мере их поступления — без ожидания накопления целого блока, что даёт низкую задержку и высокую скорость.

2.2. Принцип псевдослучайности ключевого потока

Гамма должна быть статистически неотличима от истинно случайной последовательности: равномерное распределение нулей и единиц, отсутствие коротких повторов, непредсказуемость следующих битов по известному префиксу.

Назначение: криптографическая стойкость - статистические закономерности в гамме позволяют нарушителю анализировать шифртекст и восстанавливать структуру открытого текста.

2.3. Принцип зависимости ключевого потока от секретного ключа

Гамма определяется секретным ключом: $z = G(k,\ IV)$. Без знания k воспроизвести корректный поток невозможно.

Назначение: обеспечивает секретность (принцип Керкгофса - стойкость системы определяется секретностью ключа, а не алгоритма) и управляемость: один алгоритм обслуживает множество пользователей и сеансов.

2.4. Принцип использования инициализирующего вектора

При одном и том же ключе k разные значения IV порождают различные ключевые потоки.

Назначение: предотвращает повтор гаммы без смены ключа, обеспечивает разделение сеансов и масштабируемость протоколов.

Важное условие (запрет повтора):

Если два сообщения зашифрованы одной и той же гаммой, $c1 = m1 \oplus z$, $c2 = m2 \oplus z$, то $c1 \oplus c2 = m1 \oplus m2$. Это раскрывает связь между открытыми текстами. Следовательно, для фиксированного k повтор IV недопустим.

2.5. Принцип большого периода ключевой последовательности

Генератор не должен зацикливаться на коротком фрагменте: период последовательности должен быть практически неограниченным в рамках одного сеанса.

Назначение: защита длинных сообщений и долгих сессий — короткий период позволяет нарушителю восстановить структуру гаммы статистически.

Замечание: большой период сам по себе не гарантирует безопасность - необходима также криптографическая непредсказуемость (PRG-свойства).

2.6. Принцип нелинейности преобразований

Внутренние функции генератора (булевы функции, фильтры, комбинирующие функции) должны быть нелинейными; чисто линейные схемы (например, LFSR без нелинейного фильтра) поддаются алгебраическому анализу.

Назначение: стойкость к линейному и корреляционному анализу - нелинейность разрушает простые соотношения между внутренним состоянием S_t, ключом k и выходом z.

2.7. Принцип сложности восстановления внутреннего состояния

Поточный шифр имеет внутреннее состояние S_t, которое изменяется на каждом шаге: S_{t+1} = F(S_t, k), z_t = H(S_t). При известном открытом тексте нарушитель может извлечь фрагменты z по формуле z_i = m_i ⊕ c_i. Требование: даже знание фрагмента гаммы не должно позволять восстановить k или всё состояние генератора.

Назначение:

- защита ключа от восстановления по выходу;
- forward security - знание части z не позволяет предсказывать будущие биты;
- backward privacy - компрометация текущего S_t не раскрывает прошлую гамму.

2.8. Принцип синхронизации отправителя и получателя

Для правильного расшифрования отправитель и получатель должны использовать одинаковый ключевой поток. Это требует синхронизации генераторов гаммы.

Различают два основных типа поточных шифров:

Синхронные поточные шифры - ключевой поток формируется независимо от шифртекста.

Самосинхронизирующиеся поточные шифры - ключевой поток зависит от предыдущих символов шифртекста.

Назначение:

- Корректность расшифрования - стороны должны породить один и тот же поток z.
- Устойчивость к потерям/ошибкам - самосинхронизирующиеся схемы могут восстанавливаться после потерь.
- Контроль ошибок - поведение ошибок различается и определяется типом шифра.
- Практическая применимость - тип выбирают по свойствам канала/протокола.

2.9. Принцип эффективности реализации

Архитектура и примитивы (операции типа add-rotate-XOR) должны эффективно работать как в программной, так и в аппаратной среде, включая встроенные и мобильные устройства.

Назначение: высокая скорость, низкая задержка, экономия ресурсов — поточные шифры применяются именно там, где эти параметры критичны (аудио/видео, телекоммуникации).

2.10. Принцип достаточной длины ключа

Пространство ключей |K| должно быть достаточно большим, чтобы полный перебор был вычислительно нецелесообразен.

Назначение: стойкость к атаке перебором с учётом роста вычислительных мощностей; длина ключа подбирается в соответствии с актуальными стандартами безопасности.

2.11. Принцип корректной инициализации генератора

Перед началом выработки ключевого потока генератор должен быть правильно инициализирован ключом и начальным значением. Часто после загрузки k и IV выполняют «прогрев» состояния.

Формально: начальное состояние $S_0 = Init(k, IV)$, далее генерация $z_t = H(S_t)$.

Назначение: скрытие связи между k, IV и первыми битами z; исключение слабых начальных состояний, при которых начальный фрагмент гаммы мог бы быть предсказуем.

2.12. Принцип разделения функций шифрования и контроля целостности

Поточный шифр обеспечивает конфиденциальность, но не целостность: изменение бита c_i предсказуемо меняет бит m_i. Защита от модификаций требует дополнительного механизма (MAC, AEAD).

Назначение: предотвращение активных атак (незаметной подмены данных); на практике применяют AEAD-режимы, например ChaCha20-Poly1305 (RFC 8439), или схему «Encrypt-then-MAC».

Вывод

Поточные системы шифрования строятся на принципе генерации ключевого потока и его наложения на открытый текст. Основная формула: $c = m \oplus z$, где z - гамма, формируемая генератором на основе секретного ключа и, как правило, инициализирующего вектора.

Совокупность изложенных принципов обеспечивает три группы свойств:

- Стойкость - псевдослучайность, нелинейность, сложность восстановления состояния, достаточная длина ключа, запрет повтора гаммы;
- Корректность - синхронизация, корректная инициализация, контроль целостности;
- Эффективность - потоковая обработка, оптимизированные примитивы, минимальное распространение ошибок.

Список литературы

1. Шнайер Б. Прикладная криптография. - М.: ТРИУМФ, 2012.

2. Молдовян А.А. и др. Криптография: скоростные шифры. - СПб.: БХВ-Петербург, 2002.

3. Масленников М.Е. Практическая криптография. - СПб.: БХВ-Петербург, 2003.

4. Алферов А.П. и др. Основы криптографии. - М.: Гелиос АРВ, 2001.

5. Смарт Н. Криптография. - М.: Техносфера, 2005.

6. Menezes A.J., van Oorschot P.C., Vanstone S.A. Handbook of Applied Cryptography. - CRC Press, 2001.

7. Ященко В.В. (ред.) Введение в криптографию. - М.: МЦНМО, 2012.

8. Мао В. Современная криптография: теория и практика. - М.: Вильямс, 2005.

Вопрос 4. Приведите пример режима шифрования, в котором биективность всех шифрующих отображений базового режима не является необходимой для однозначного расшифрования криптограмм. Ответ обоснуйте.

1. Общая идея режима CTR

Режим CTR превращает блочный шифр или вообще некоторую детерминированную функцию в генератор ключевого потока.

Для каждого блока формируют вход генератора: $X_i = Nonce \ // \ counter_i$ (конкатенация $//$).

Затем вычисляют ключевой блок (гамму): $S_i = E_k(X_i)$.

После этого шифруют: $C_i = P_i \oplus S_i$.

Обозначения:

- P_i - i-й блок открытого текста;
- C_i - i-й блок шифртекста;
- S_i - i-й блок ключевого потока;
- E_k - базовое шифрующее преобразование на ключе k;
- $\oplus$ - побитовое XOR.

2. Механизм расшифрования в CTR

Получатель повторяет генерацию ключевого потока: $S_i = E_k(Nonce \ // \ counter_i)$ и восстанавливает текст: $P_i = C_i \oplus S_i$.

Обоснование: $P_i = (P_i \oplus S_i) \oplus S_i$, а потому что $S_i \oplus S_i = 0$, имеем $P_i = P_i \oplus 0 = P_i$ — восстановление однозначно.

3. Почему биективность E_k не требуется

В режимах ECB/CBC нужно инвертировать шифр: $D_k = E_k^{-1}$ (например, $C_i = E_k(P_i)$, $P_i = D_k(C_i)$), иначе расшифрование невозможно.

В CTR базовое отображение применяется не к открытому тексту, а к счётчику/синхропосылке: $S_i = E_k(Nonce \ // \ counter_i)$, а текст получается XOR-ом: $C_i = P_i \oplus S_i$.

Для расшифрования не нужен E_k^{-1} — достаточно повторно вычислить тот же S_i. Следовательно, для однозначности достаточно знать k, Nonce, порядок $counter_i$ и чтобы E_k была детерминированной. Биективность E_k не обязательна.

4. Обоснование на формальном уровне

Пусть $E_k: \{0,1\}^n \to \{0,1\}^n$ не биективна (есть $X_a \neq X_b$ с $E_k(X_a) = E_k(X_b)$). В CTR открытый текст не извлекается как $D_k(C_i)$, а вычисляется как $P_i = C_i \oplus S_i$, где фиксирован $S_i = E_k(Nonce \ // \ counter_i)$.

При фиксированном S_i отображение $P_i \mapsto P_i \oplus S_i$ — биекция, поэтому для каждого C_i ровно один $P_i = C_i \oplus S_i$. Однозначность обеспечивается обратимостью XOR, а не биективностью E_k.

5. Важное уточнение о стойкости

Отсутствие требования биективности относится только к однозначности расшифрования. Для стойкости необходимы:

- непредсказуемость и неповторяемость ключевого потока;
- уникальные Nonce и монотонный $counter_i$ под данным ключом k;

- криптографическая стойкость базового преобразования. При повторе гаммы: $C_1 = P_1 \oplus S$, $C_2 = P_2 \oplus S \Rightarrow C_1 \oplus C_2 = P_1 \oplus P_2$.

6. Итоговое обоснование

В CTR биективность базового отображения не требуется для корректного и однозначного восстановления открытого текста, потому что расшифрование - это XOR с детерминированно воспроизводимым ключевым потоком, а не обращение E_k.

Аналогичный аргумент применим к режиму OFB: ключевой поток порождается как итерация E_k, а расшифрование также выполняется XOR-ом; потому биективность E_k для однозначности не требуется.

Список литературы

- Dworkin, M. Recommendation for Block Cipher Modes of Operation: Methods and Techniques. NIST SP 800-38A, 2001.
- Menezes, A.J., van Oorschot, P.C., Vanstone, S.A. Handbook of Applied Cryptography. CRC Press, 2001.

Вопрос 5. Сколько 64-битовых блоков открытого текста искажается при расшифровании криптограммы симметричного блочного шифра, если в шифрованном тексте исказился один бит и используется режим гаммирования с обратной связью по шифртексту, в котором блоки открытого и шифрованного текста имеют длину 8 бит? Ответ обоснуйте.

1. Логика работы режима CFB

Режим CFB - Cipher Feedback Mode, или режим гаммирования с обратной связью по шифртексту. В этом режиме блочный шифр используется не напрямую для шифрования блока открытого текста, а для выработки гаммы (Z_i), которая затем складывается с открытым текстом по XOR.

Пусть задано: размер блока базового шифра $n = 64$ бита; размер сегмента $s = 8$ бит; R_i - 64-битовый регистр; E_K - шифрование базового шифра; P_i - i-й 8-битовый сегмент открытого текста; C_i - i-й 8-битовый сегмент шифртекста; Z_i - 8-битовая гамма.

1.1. Шифрование в CFB

На каждом шаге: $Y_i = E_K(R_i)$; берутся старшие s бит: $Z_i = MSB_s(Y_i)$ (в нашей задаче $s = 8$); затем $C_i = P_i \oplus Z_i$; регистр обновляется сдвигом на s бит с заведением C_i: $R_{i+1} = shift(R_i, C_i)$.

1.2. Расшифрование в CFB

Используются те же операции шифрования базового шифра: $Y_i = E_K(R_i)$, $Z_i = MSB_s(Y_i)$, далее $P_i = C_i \oplus Z_i$, и регистр обновляется так же: $R_{i+1} = shift(R_i, C_i)$. Ошибочный C_i попадает в регистр и некоторое число шагов влияет на последующую гамму.

2. Как распространяется ошибка в CFB

Пусть в одном 8-битовом сегменте шифртекста C_j исказился один бит, то есть при расшифровании поступает $C'_j = C_j \oplus e$, где e - 8-битовая маска с единицей в одном бите.

2.1. Ошибка в текущем сегменте открытого текста

При расшифровании текущего сегмента используется корректная гамма Z_j (поскольку R_j ещё не содержит ошибочного шифртекста): $P'_j = C'_j \oplus Z_j = (C_j \oplus e) \oplus Z_j = (C_j \oplus Z_j) \oplus e = P_j \oplus e$. Искажается ровно тот бит текущего 8-битового сегмента P_j, который искажён в C_j.

2.2. Ошибка в следующих сегментах

После обработки C'_j ошибочный сегмент попадает в регистр обратной связи. Так как длина регистра $n = 64$ бита, а на каждом шаге он сдвигается на $s = 8$ бит, ошибочный 8-битовый сегмент находится в регистре ровно $n/s = 8$ следующих шагов. Следовательно, ошибка влияет на формирование гаммы для сегментов $P_{j+1}, \ldots, P_{j+8}$. После этого ошибочный сегмент полностью вытесняется из регистра, и начиная с P_{j+9} расшифрование снова корректно.

3. Общее число искажённых 8-битовых сегментов

Искажаются: текущий сегмент P_j и следующие $n/s = 8$ сегментов $P_{j+1}, \ldots, P_{j+8}$. Итого $1 + 8 = 9$ сегментов по 8 бит, то есть $9 \cdot 8 = 72$ бита.

4. Переход к 64-битовым блокам открытого текста

Один 64-битовый блок открытого текста содержит $64/8 = 8$ 8-битовых сегментов. Поскольку искажено 9 сегментов (72 бита), они занимают $\text{ceil}(72/64) = 2$ блока. Следовательно, затронуты два соседних 64-битовых блока.

Иллюстрация

Пусть 64-битовый блок открытого текста состоит из 8 байтов: B1, B2, B3, B4, B5, B6, B7, B8. Если ошибка пришлась на сегмент, соответствующий байту B3, то искажаются: в текущем блоке - B3, B4, B5, B6, B7, B8 (6 байтов); в следующем блоке — B1, B2, B3 (3 байта). Всего: 6 + 3 = 9 байтов.

5. Итоговое обоснование

В режиме CFB ошибка в одном бите шифртекста влияет: (a) на соответствующий 8-битовый сегмент открытого текста напрямую, и (b) на следующие сегменты через регистр обратной связи. Поскольку размер регистра равен размеру блока (64 бита), а сдвиг - 8 бит, ошибочный сегмент остаётся в регистре $64/8 = 8$ шагов. Поэтому всего искажается $1 + 8 = 9$ сегментов (72 бита), что затрагивает $\text{ceil}(72/64) = 2$ 64-битовых блока.

Ответ

При искажении одного бита шифртекста в режиме CFB с 64-битовым блоком базового шифра и 8-битовыми сегментами при расшифровании искажаются 2 64-битовых блока открытого текста. В более детальной форме: искажается 9 байтов (9 сегментов по 8 бит), что соответствует двум соседним 64-битовым блокам.

Список литературы

- Dworkin, M. Recommendation for Block Cipher Modes of Operation: Methods and Techniques. NIST Special Publication 800-38A, 2001.

- Menezes, A.J., van Oorschot, P.C., Vanstone, S.A. Handbook of Applied Cryptography. CRC Press, 2001.

Вопрос 6. Однонаправленная хэш-функция принимает значения из множества двоичных n-мерных векторов. При каком n случайная пара сообщений составляет коллизию с вероятностью, не большей 10^-9? Приведите ход решения.

Краткий ответ
Минимальное n = 30.

Модель и обозначения
- Предполагаем идеализированную хэш-функцию с равномерным распределением значений на множестве H = {0,1}^n (модель случайной функции/идеального хэша).
- Рассматриваем ровно две независимые случайные строки сообщений M1 и M2; интересует событие коллизии h(M1) = h(M2).

Шаг 1. Вероятностная модель и базовая формула

Для двух независимых сообщений вероятность коллизии равна вероятности совпадения двух независимых равномерных n-битовых значений:

P[collision for 2 messages] = 1 / 2^n.

Шаг 2. Требуемое неравенство

По условию требуется, чтобы эта вероятность не превышала 10^-9:

1 / 2^n ≤ 10^-9.

Шаг 3. Выражаем n через логарифмы по основанию 2

Берём log2 от обеих частей (учитывая монотонность логарифма):

-n ≤ log2(10^-9).

Преобразуем правую часть: log2(10^-9) = -9 · log2(10).

Следовательно,

n ≥ 9 · log2(10).

Численное значение log2(10) возьмём с достаточной точностью: log2(10) ≈ 3.321928094.

Тогда

n ≥ 9 · 3.321928094 ≈ 29.897352846.

Поскольку n - целое число бит, берём наименьшее целое, не меньшее правой части:

n_min = 30.

Шаг 4. Проверка (sanity check)

- Для n = 30: 2^-30 ≈ 9.3132 · 10^-10 ≤ 10^-9 - условие выполняется.
- Для n = 29: 2^-29 ≈ 1.8626 · 10^-9 > 10^-9 - условие нарушается, значит n = 29 недостаточно.

Итак, минимальное n действительно равно 30.

Замечание про «парадокс дней рождений»

- Для пары сообщений используется точная формула 1/2^n, и парадокс дней рождений не требуется.
- Для множества из t сообщений вероятность хотя бы одной коллизии (при малых вероятностях) аппроксимируется «правилом дней рождений»:

P_collision ≈ t(t−1) / 2^{n+1}.

Из неё видно, что для «поиска коллизии» в общем случае нужна примерно 2^{n/2} работа, что объясняет выбор «двойной» длины хэша при целевой стойкости к коллизиям (например, 256 бит → ≈ 128-битная стойкость).

Итог

- Требуемое условие 1/2^n ≤ 10^-9 даёт n ≥ 9·log2(10) ≈ 29.897....
- Минимальное целое n, удовлетворяющее условию, - 30 бит.

Список литературы

- Menezes, van Oorschot, Vanstone. Handbook of Applied Cryptography, §9 (хэш-функции и вероятностные оценки).
- Stinson, D. Cryptography: Theory and Practice (оценки вероятностей коллизий и «birthday bound»).